Команда Сбера

Протоколы взаимодействия AI-агентов

Тращенков Сергей

Разработка и внедрение AI-агентов · МФТИ

Материалы

Telegram-канал

Карта взаимодействий

Помощник для поездки обращается к инструментам, другому агенту и пользовательскому приложению.

MCP

Инструменты и контекст

A2A

Задачи удалённому агенту

AG-UI / A2UI

События и интерфейс

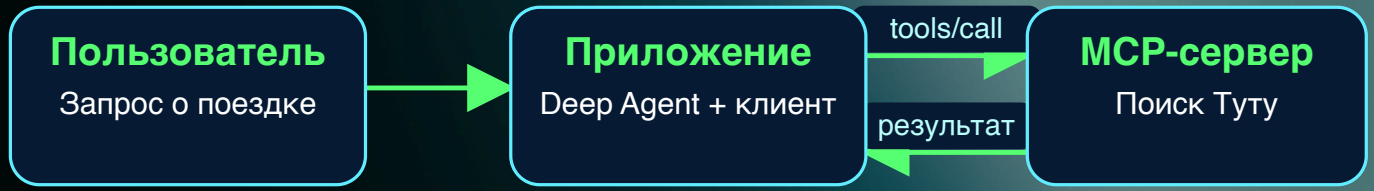
ACP связывает редактор и агентный процесс. Skills упаковывают инструкции и файлы.

Материалы

Telegram-канал

Кто выполняет инструмент

Host создаёт MCP-клиента и передаёт модели описания tools. Вычисления инструмента выполняются на сервере.



Материалы

Telegram-канал

Три основных примитива MCP

Tools

Операции с аргументами и результатом.

`search_rail`

Resources

Адресуемые данные для приложения.

`course://overview`

Prompts

Шаблоны сообщений с параметрами.

`prepare_lecture_brief`

Приложение получает ресурсы и prompts, собирает контекст и предоставляет tools модели.

Материалы

Telegram-канал

Цикл вызова инструмента

Модель

Имя tool и аргументы

→

Приложение

Отправляет MCP-запрос

→

Сервер

Выполняет операцию

Результат возвращается в ToolMessage. Следующий шаг модели использует эти данные.

Ресурс становится контекстом

Resource

Описание курса по URI

→

Host

Читает содержимое

→

Сообщение модели

Добавляет нужный текст

URI определяет источник данных. Решение о включении данных в запрос принимает приложение.

Prompt задаёт структуру задачи

Аргументы

```
topic_name="mcp"
```

```
audience="студенты"
```

Результат сервера

Сообщения с задачей подготовки справки и требованиями к ответу

Получение prompt формирует сообщения. Запуск модели выполняется отдельным шагом.

Сборка входа модели

Описание курса, шаблон справки и инструменты чтения учебных файлов участвуют в одном опыте.



Материалы

Telegram-канал

Редакции MCP и версии SDK

Спецификация

`2025-11-25`
↓
`2026-07-28`

Python SDK

`v1` → `v2`

Отдельная нумерация библиотеки

> Обновление протокола и обновление Python API обсуждаем отдельно.

Начало обмена в двух редакциях

2025-11-25	2026-07-28
initialize и согласование возможностей	Версия и возможности в _meta запроса
HTTP-сессия Mcp-Session-Id	Явные идентификаторы объектов в аргументах
Сведения при инициализации	server/discover и метаданные запроса

В ноутбуке вызываем tools/call без предварительного initialize, затем пробуем старое рукопожатие.

Дополнительный ввод: MRTR

Сервер

Возвращает input_required

→

Приложение

Получает ввод пользователя

→

Повтор запроса

Передаёт inputResponses

Дополнительно: subscriptions/listen; Tasks как расширение; Roots/Sampling/Logging — устаревающие возможности.

Материалы Telegram-канал

Python SDK v1 → v2

Сервер

`FastMCP` → `MCPServer`

Параметры транспорта передаются в run()

Клиент

```
from mcp import Client
```

Async context manager держит соединение открытым

Декораторы tools, resources и prompts сохраняют знакомую структуру.

MCP в экосистеме LangChain

Прежнее подключение	Используемое подключение
langchain-mcp-adapters	langchain[mcp]
MultiServerMCPClient	MCPAdapter
get_tools()	await adapter.list_tools()

MCPAdapter работает как async context manager. Resources и prompts читаем клиентом SDK. Новый namespace находится в beta.

Материалы

Telegram-канал

Каталог Туту без вызова модели

Подключение

URL публичного MCP

→

Обнаружение

list_tools()

→

Проверка

Имена, описания, схемы

До создания агента приложение уже знает доступные операции и их параметры.

JSON Schema связывает клиента и модель

Обязательные поля

Города, дата и другие условия задаются схемой конкретного инструмента

Необязательные поля

```
anyOf: [integer, null]
```

Совместимость проверяется на всём пути к API

В примерах включён GigaChat API v2. Исходные схемы MCP передаются без упрощения.

Материалы

Telegram-канал

Сборка и запуск Deep Agent

```
tools = await adapter.list_tools()
agent = create_deep_agent(model=model, tools=tools)
result = await agent.ainvoke({"messages": messages})
```

> Приложение предоставляет инструменты. Во время запуска модель выбирает конкретные вызовы.

История одного поиска

AIMessage

search_rail и аргументы

→

ToolMessage

Ответ Туту

→

AIMessage

Поезд, время и цена

Проверяем дату, направление, прямой маршрут и соответствие ответа полученным данным.

Свой сервер учебных материалов

Resources

Описание курса
Сведения о темах

Prompt

Справка по теме
Заданная аудитория

Tools

Список файлов
Чтение начала файла

> Один сервер демонстрирует все три примитива на данных одной учебной папки.

stdio: сервер как дочерний процесс

MCP-клиент

Запускает Python

→

stdin / stdout

Сообщения JSON-RPC

→

Сервер

Читает учебные файлы

Async context manager управляет соединением и завершением дочернего процесса.

Материалы Telegram-канал

Полный опыт: справка по теме

До запуска

Resource + prompt

→

Во время запуска

Список файлов → чтение

→

После запуска

Справка и история

На экране видны сформированные входные сообщения и tool calls агента.

Материалы

Telegram-канал

HTTP: тот же набор возможностей

Публикуем тот же сервер через Streamable HTTP и рассматриваем фактический запрос.

Запрос

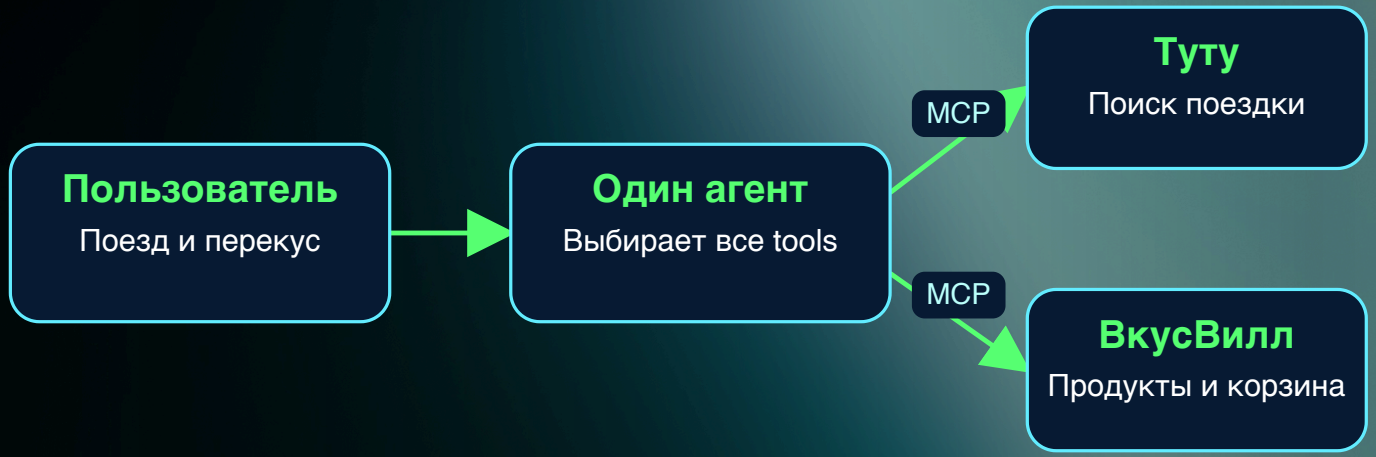
Метод tools/call
Аргументы операции
Версия и возможности в _meta

Ответ

Результат операции
resultType
Метаданные сервера

Streamable HTTP может передавать SSE. Отдельный старый транспорт HTTP+SSE помечен устаревающим.

Один агент и два MCP-сервера



Координатор сам выбирает инструменты поиска поездки и продуктов.

Данные между инструментами ВкусВилла

Поиск

Названия и id продуктов

→

Выбор

3–4 позиции перекуса

→

Корзина

products: id + количество

Получаем ссылку на корзину. Оформление и оплата заказа в опыте не выполняются.

OAuth: разрешение пользователя

Приложение

Зарегистрированный OAuth-клиент

→

Пользователь

Подтверждает доступ

→

Сервис

Выдаёт ограниченный доступ

OAuth 2.0 + PKCE. Персональные сведения требуют согласия; в практике используются публичные операции.

Практика: расширить свой сервер

Добавить

Tool подсчёта учебных файлов по формату

Проверить

list_tools → ручной вызов → вызов агентом

Доступ к файлам остаётся ограничен папкой data/course.

Команда Сбера

Взаимодействие агентов и приложений

Продолжение · 45 минут

§ 9–17

Материалы

Telegram-канал

Два агента с разной ответственностью



Удалённый агент самостоятельно ищет поездку. Координатор объединяет результат с подбором продуктов.

Agent Card: как обратиться к агенту

Идентичность

Имя и описание
Версия реализации

Интерфейс

URL
JSONRPC,
protocolVersion

Возможности

Skills
Форматы сообщений
Capabilities

Skills в Agent Card описывают возможности агента. Пакеты SKILL.md рассматриваются отдельно.

Сообщение, задача и artifact

SendMessage

Запрос пользователя

→

Task

Статус выполнения

→

Artifacts

Полученные результаты

В примере ждём завершённую задачу. Историю, отмену и все режимы A2A отдельно не сравниваем.

Одна фабрика — два способа публикации

Общая логика

Deep Agents + GigaChat
Один system prompt
Одинаковые MCP tools Туту

Две реализации

Сервер на a2a-sdk

LangGraph Agent Server

> Изменяется серверная обвязка. Запрос и клиент остаются одинаковыми.

Способ A: a2a-sdk

Agent Card

Описываем интерфейс

→

AgentExecutor

Запускаем общий граф

→

TaskUpdater

Статус и artifact

Код сервера явно показывает перевод A2A-задачи в запуск агента и обратно.

Материалы

Telegram-канал

Способ B: LangGraph Agent Server

```
from agents.travel import travel_graph

graph = travel_graph
```

```
{"graphs": {"travel": "./servers/agent_server.py:graph"}}
```

> langgraph dev предоставляет A2A endpoint и управление запуском. Фабрика держит MCP-соединение открытым.

Материалы Telegram-канал

Сравнение одинаковой задачи

Проверяем	Оба варианта
Фабрика агента	Одна и та же функция
Источник данных	MCP Туту
Вход	Одинаковый запрос о поездке
Выход	Завершённая задача и текстовый artifact

Цена и текст могут отличаться между запросами. Сравниваем выполнение задачи и корректность данных.

Клиент и сервер: совместимость форматов

Наблюдение

Строгий protobuf-клиент отклоняет дополнительное поле kind у Agent Server

Учебный клиент

Явный JSON-RPC
Читает статус и текстовый artifact обоих серверов

Совпадение номера версии в карточке проверяем реальным обменом.

Координатор переключается между серверами

travel_agent

A2A-запрос о поездке

→

MCP ВкусВилла

Поиск и ссылка на корзину

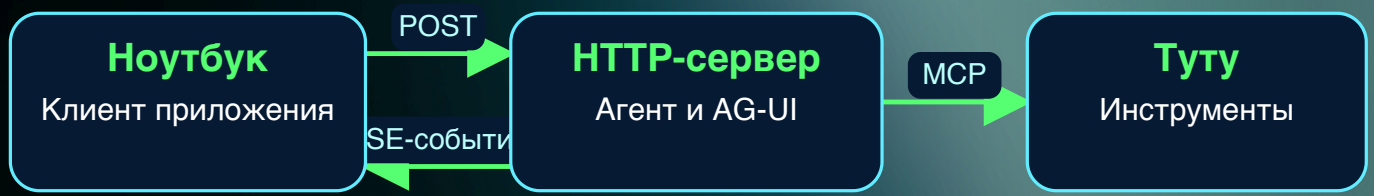
→

Ответ

Поезд + перекус

Меняется карточка подключения удалённого агента. Логика координатора сохраняется.

AG-UI: события для приложения

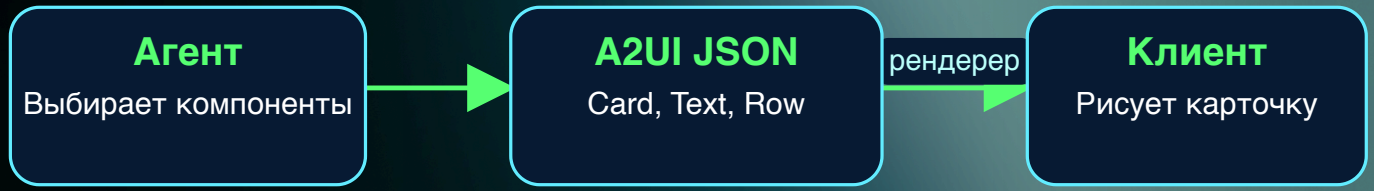


Ноутбук играет роль клиента. HTTP-сервер с агентом запускается отдельным процессом.

Что показывает поток событий

Событие	Действие клиента
RUN_STARTED	Индикатор работы
TOOL_CALL_START / RESULT	Операция и полученные сведения
TEXT_MESSAGE_CONTENT	Дописывание текста
RUN_FINISHED / ERROR	Завершение или сообщение об ошибке

A2UI: описание интерфейса



Агент выбирает компоненты из каталога. Клиент отвечает за их отображение и обработку действий.

Карточка поезда в JSON

Структура

createSurface
updateComponents
root → Card → Text

Данные

Номер поезда
Отправление и прибытие
Цена

В опыте используется v0.9. Ноутбук показывает JSON и статический просмотр базовых компонентов.

Материалы

Telegram-канал

ACP: редактор и агентный процесс



В демонстрации ноутбук выполняет роль минимального клиента. Сетевой порт не требуется.

Сессия ACP по шагам

initialize

Версия и возможности

→

session/new

Рабочий каталог

→

session/prompt

Запрос пользователя

Поступают session/update, затем stopReason=end_turn. stdout содержит JSON-RPC, диагностика идёт в stderr.

Материалы Telegram-канал

Коммерция: участники и разрешения

Пользователь

Задаёт условия и согласие

→

Агент и продавец

Уточняют покупку

→

Платёжная сторона

Проверяет разрешение

Разбор теоретический. Финансовые операции в ноутбуке не выполняются.

Материалы Telegram-канал

Разные части коммерческого процесса

Протокол	Назначение в обзоре
Agentic Commerce Protocol	Checkout между агентом и продавцом
UCP	Поиск, корзина, оформление, работа с заказом
AP2	Проверяемые полномочия и разрешение оплаты
x402	Оплата доступа к HTTP-ресурсу

Agent Client Protocol и Agentic Commerce Protocol имеют одинаковую аббревиатуру ACP.

Материалы

Telegram-канал

Контрольные вопросы

Инструменты и контекст

Кто читает resource?
Что возвращает prompt?
Где исполняется tool?

Агенты и приложение

Кто выполняет поиск по A2A?
Кто рисует A2UI?
Что передаёт stdout ACP?

Команда Сбера

Skills: дополнительный материал

Инструкции, вспомогательные файлы и проверяемый результат

Материалы

Telegram-канал

Папка навыка

SKILL.md

Имя и описание
Процедура выполнения
Требования

Файлы

qr_generate.py
qr_read.py
Метаданные источника

Агент выбирает навык по описанию, затем читает подробные инструкции.

QR: результат можно проверить



Агент читает SKILL.md, запускает оба скрипта и сравнивает строки. Python независимо повторяет проверку.

Загрузка навыка в LangChain

list_skills

Краткие описания

→

load_skill

Полный SKILL.md

→

Агент

План выполнения

Во втором примере показываем выбор и загрузку инструкции. Исполнение скриптов разбирается через Deep Agents.

Материалы

Telegram-канал

QR-зависимости на разных ОС

ОС	Нативная библиотека
Windows x64	DLL в pyzbar; при ошибке — Visual C++ Runtime 2013 x64
macOS	brew install zbar
Debian/Ubuntu	apt-get install libzbar0

uv sync устанавливает Python-пакеты. check_env.py --qr проверяет создание и чтение до запуска модели.

Источники спецификаций

MCP и инструменты

- Спецификация MCP
- MCP Python SDK v2
- MCP в LangChain

Агенты и интерфейсы

- Agent2Agent
- AG-UI
- A2UI
- Agent Client Protocol

Источники коммерческого обзора

- Agentic Commerce Protocol
- Universal Commerce Protocol
- Agent Payments Protocol
- x402

> Блокнот содержит выполняемые примеры и документацию использованных библиотек.

Материалы Telegram-канал